Guía integradora: combinando listas, diccionarios, funciones, condicionales y ciclos en Python

1. ¿Por qué combinar herramientas en Python?

Cuando aprendemos Python, normalmente vemos los temas por separado:

- Listas
- Diccionarios
- Funciones
- Condicionales
- Ciclos

Pero en los programas reales casi nunca se usan de manera aislada.

Por ejemplo, si queremos crear un sistema simple para registrar estudiantes, necesitamos combinar varias herramientas:

Herramienta	¿Para qué sirve en el programa?
Lista	Guardar varios estudiantes
Diccionario	Representar los datos de cada estudiante
Función	Separar tareas del programa
Condicional	Tomar decisiones
Ciclo	Repetir acciones

Ejemplo general:

```
estudiantes = []

estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}

estudiantes.append(estudiante)
```

Aquí ya estamos combinando una lista con un diccionario.

La lista guarda muchos estudiantes.

Cada estudiante se representa con un diccionario.

---

2. Recordatorio rápido de cada herramienta

Lista

Una lista permite guardar varios elementos.

nombres = ["Ana", "Luis", "Camila"]

Se usa cuando necesitamos almacenar muchos datos del mismo tipo o relacionados.

Diccionario

Un diccionario permite guardar datos usando claves y valores.

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}
```

Se usa cuando queremos representar una entidad con varios datos.

Condicional

Un condicional permite tomar decisiones.

```
edad = 20

if edad >= 18:
    print("Es mayor de edad")
```

```
else:
    print("Es menor de edad")
```

Ciclo

Un ciclo permite repetir instrucciones.

```
for i in range(5):
    print("Hola")
```

También podemos recorrer listas:

```
nombres = ["Ana", "Luis", "Camila"]

for nombre in nombres:
    print(nombre)
```

Función

Una función permite organizar el código en bloques reutilizables.

```
def saludar(nombre):
    print("Hola", nombre)

saludar("Ana")
```

---

3. Combinando listas y ciclos

Una de las combinaciones más comunes es usar listas con ciclos.

Ejemplo:

```
nombres = ["Ana", "Luis", "Camila"]

for nombre in nombres:
    print(nombre)
```

Salida:

Ana
Luis
Camila

Aquí ocurre lo siguiente:

1. La lista `nombres` guarda varios datos.
2. El ciclo `for` recorre cada elemento.
3. En cada vuelta, la variable `nombre` toma un valor diferente.

Primero vale `"Ana"`, luego `"Luis"` y finalmente `"Camila"`.

---

4. Combinando listas y condicionales

Podemos recorrer una lista y aplicar condiciones.

Ejemplo: mostrar solo los números mayores a 10.

```
numeros = [4, 15, 8, 23, 7, 12]

for numero in numeros:
    if numero > 10:
        print(numero)
```

Salida:

```
15
23
12
```

En este caso:

- El ciclo recorre todos los números.
- El condicional revisa si cada número es mayor a 10.
- Solo se muestran los números que cumplen la condición.

---

5. Combinando listas, ciclos y contadores

Un contador es una variable que aumenta cuando ocurre algo.

Ejemplo: contar cuántos números son mayores a 10.

```
numeros = [4, 15, 8, 23, 7, 12]

contador = 0

for numero in numeros:
    if numero > 10:
        contador = contador + 1

print("Cantidad de números mayores a 10:", contador)
```

Salida:

```
Cantidad de números mayores a 10: 3
```

La variable `contador` comienza en `0`.

Cada vez que un número es mayor a 10, el contador aumenta en 1.

---

6. Combinando listas, ciclos y acumuladores

Un acumulador es una variable que va sumando valores.

Ejemplo: sumar todas las notas de una lista.

```
notas = [5.5, 6.0, 4.8, 7.0]

suma = 0

for nota in notas:
    suma = suma + nota

promedio = suma / len(notas)

print("Promedio:", promedio)
```

Salida:

Promedio: 5.825

La variable `suma` acumula el total de las notas.

Luego se divide por la cantidad de elementos de la lista.

---

7. Combinando diccionarios y condicionales

Podemos usar condicionales para revisar datos dentro de un diccionario.

Ejemplo:

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}

if estudiante["nota"] >= 4.0:
    print("Estudiante aprobado")
else:
    print("Estudiante reprobado")
```

Salida:

Estudiante aprobado

Aquí el condicional evalúa el valor de la clave `"nota"`.

---

8. Combinando diccionarios y funciones

Podemos crear funciones que reciban diccionarios.

Ejemplo:

```
def mostrar_estudiante(estudiante):
    print("Nombre:", estudiante["nombre"])
```

```
    print("Edad:", estudiante["edad"])
    print("Nota:", estudiante["nota"])

estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}

mostrar_estudiante(estudiante)
```

Salida:

```
Nombre: Ana
Edad: 20
Nota: 6.5
```

La función recibe un diccionario y muestra sus datos.

Esto permite reutilizar el mismo código para distintos estudiantes.

---

9. Combinando listas y diccionarios

Una combinación muy importante es tener una lista de diccionarios.

Ejemplo:

```
estudiantes = [
    {
        "nombre": "Ana",
        "edad": 20,
        "nota": 6.5
    },
    {
        "nombre": "Luis",
        "edad": 22,
        "nota": 5.8
    },
    {
        "nombre": "Camila",
        "edad": 19,
        "nota": 6.9
    }
```

]

En este caso:

- `estudiantes` es una lista.
- Cada elemento de la lista es un diccionario.
- Cada diccionario representa un estudiante.

Esta estructura se usa mucho en programación real.

Por ejemplo:

- Lista de usuarios.
- Lista de productos.
- Lista de ventas.
- Lista de pacientes.
- Lista de cursos.
- Lista de registros desde una base de datos.

---

10. Recorrer una lista de diccionarios

Podemos recorrer la lista y acceder a los datos de cada diccionario.

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "nota": 6.5},
    {"nombre": "Luis", "edad": 22, "nota": 5.8},
    {"nombre": "Camila", "edad": 19, "nota": 6.9}
]

for estudiante in estudiantes:
    print("Nombre:", estudiante["nombre"])
    print("Edad:", estudiante["edad"])
    print("Nota:", estudiante["nota"])
    print("----------")
```

Salida:

```
Nombre: Ana
Edad: 20
Nota: 6.5
----------
Nombre: Luis
Edad: 22
```

Nota: 5.8

Nombre: Camila
Edad: 19
Nota: 6.9

El ciclo recorre cada estudiante.

En cada vuelta, la variable `estudiante` contiene un diccionario diferente.

---

11. Usar condicionales dentro de una lista de diccionarios

Ejemplo: mostrar solo los estudiantes aprobados.

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "nota": 6.5},
    {"nombre": "Luis", "edad": 22, "nota": 3.8},
    {"nombre": "Camila", "edad": 19, "nota": 6.9}
]

for estudiante in estudiantes:
    if estudiante["nota"] >= 4.0:
        print(estudiante["nombre"], "está aprobado")
```

Salida:

Ana está aprobado
Camila está aprobado

Aquí se combinan:

- Lista
- Diccionario
- Ciclo
- Condicional

---

12. Agregar diccionarios a una lista

Podemos crear un diccionario y agregarlo a una lista con `append()`.

```
estudiantes = []

estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}

estudiantes.append(estudiante)

print(estudiantes)
```

Salida:

```
[{'nombre': 'Ana', 'edad': 20, 'nota': 6.5}]
```

Este patrón es muy común cuando se registran datos ingresados por el usuario.

---

13. Crear estudiantes usando datos ingresados por el usuario

```
estudiantes = []

nombre = input("Ingrese nombre: ")
edad = int(input("Ingrese edad: "))
nota = float(input("Ingrese nota: "))

estudiante = {
    "nombre": nombre,
    "edad": edad,
    "nota": nota
}

estudiantes.append(estudiante)

print(estudiantes)
```

Ejemplo de ejecución:

```
Ingrese nombre: Ana
Ingrese edad: 20
Ingrese nota: 6.5
[{'nombre': 'Ana', 'edad': 20, 'nota': 6.5}]
```

---

14. Repetir el ingreso de datos con un ciclo

Ahora podemos usar un ciclo para ingresar varios estudiantes.

```
estudiantes = []

for i in range(3):
    nombre = input("Ingrese nombre: ")
    edad = int(input("Ingrese edad: "))
    nota = float(input("Ingrese nota: "))

    estudiante = {
        "nombre": nombre,
        "edad": edad,
        "nota": nota
    }

    estudiantes.append(estudiante)

print(estudiantes)
```

Este programa permite registrar 3 estudiantes.

---

15. Usar `while` para ingresar datos hasta que el usuario decida salir

El ciclo `while` es útil cuando no sabemos cuántas veces se repetirá una acción.

```
estudiantes = []
```

```
while True:
    nombre = input("Ingrese nombre o escriba salir: ")

    if nombre == "salir":
        break

    edad = int(input("Ingrese edad: "))
    nota = float(input("Ingrese nota: "))

    estudiante = {
        "nombre": nombre,
        "edad": edad,
        "nota": nota
    }

    estudiantes.append(estudiante)

print("Estudiantes registrados:")
print(estudiantes)
```

Aquí el programa sigue pidiendo estudiantes hasta que el usuario escriba `"salir"`.

---

16. Organizar el programa usando funciones

El código anterior funciona, pero puede crecer mucho.

Por eso conviene usar funciones.

Primero podemos crear una función para registrar un estudiante:

```
def crear_estudiante():
    nombre = input("Ingrese nombre: ")
    edad = int(input("Ingrese edad: "))
    nota = float(input("Ingrese nota: "))

    estudiante = {
        "nombre": nombre,
        "edad": edad,
        "nota": nota
    }

    return estudiante
```

Esta función:

1. Pide los datos.
2. Crea un diccionario.
3. Retorna el diccionario.

Luego podemos usarla así:

```
estudiantes = []

nuevo_estudiante = crear_estudiante()

estudiantes.append(nuevo_estudiante)
```

---

17. Función para mostrar estudiantes

También podemos crear una función para mostrar los estudiantes.

```
def mostrar_estudiantes(estudiantes):
    for estudiante in estudiantes:
        print("Nombre:", estudiante["nombre"])
        print("Edad:", estudiante["edad"])
        print("Nota:", estudiante["nota"])
        print("-------------")
```

Esta función recibe una lista de diccionarios y la recorre.

---

18. Función para calcular el promedio del curso

```
def calcular_promedio_curso(estudiantes):
    suma = 0

    for estudiante in estudiantes:
        suma = suma + estudiante["nota"]

    promedio = suma / len(estudiantes)
```

```
    return promedio
```

Esta función:

1. Recibe la lista de estudiantes.
2. Recorre cada estudiante.
3. Suma sus notas.
4. Calcula el promedio.
5. Retorna el resultado.

---

19. Cuidado con listas vacías

Si una lista está vacía, no podemos calcular el promedio porque estaríamos dividiendo por cero.

Por eso usamos un condicional.

```
def calcular_promedio_curso(estudiantes):
    if len(estudiantes) == 0:
        return 0

    suma = 0

    for estudiante in estudiantes:
        suma = suma + estudiante["nota"]

    promedio = suma / len(estudiantes)

    return promedio
```

Aquí se combinan:

- Función
- Condicional
- Ciclo
- Lista
- Diccionario

---

20. Función para buscar un estudiante

```
def buscar_estudiante(estudiantes, nombre_buscado):
    for estudiante in estudiantes:
        if estudiante["nombre"] == nombre_buscado:
            return estudiante

    return None
```

Esta función recorre la lista.

Si encuentra al estudiante, lo retorna.

Si no lo encuentra, retorna `None`.

Ejemplo de uso:

```
resultado = buscar_estudiante(estudiantes, "Ana")

if resultado is not None:
    print("Estudiante encontrado:", resultado)
else:
    print("Estudiante no encontrado")
```

---

21. Programa completo: sistema simple de estudiantes

Este ejemplo combina todos los elementos anteriores.

```
def mostrar_menu():
    print("===== MENÚ PRINCIPAL =====")
    print("1. Registrar estudiante")
    print("2. Mostrar estudiantes")
    print("3. Buscar estudiante")
    print("4. Mostrar promedio del curso")
    print("5. Salir")


def crear_estudiante():
    nombre = input("Ingrese nombre: ")
    edad = int(input("Ingrese edad: "))
    nota = float(input("Ingrese nota: "))

    estudiante = {
        "nombre": nombre,
```

```
        "edad": edad,
        "nota": nota
    }

    return estudiante


def mostrar_estudiantes(estudiantes):
    if len(estudiantes) == 0:
        print("No hay estudiantes registrados")
    else:
        for estudiante in estudiantes:
            print("Nombre:", estudiante["nombre"])
            print("Edad:", estudiante["edad"])
            print("Nota:", estudiante["nota"])
            print("----------")


def buscar_estudiante(estudiantes, nombre_buscado):
    for estudiante in estudiantes:
        if estudiante["nombre"] == nombre_buscado:
            return estudiante

    return None


def calcular_promedio_curso(estudiantes):
    if len(estudiantes) == 0:
        return 0

    suma = 0

    for estudiante in estudiantes:
        suma = suma + estudiante["nota"]

    promedio = suma / len(estudiantes)

    return promedio


estudiantes = []

while True:
    mostrar_menu()

    opcion = input("Seleccione una opción: ")

    if opcion == "1":
```

```
        nuevo_estudiante = crear_estudiante()
        estudiantes.append(nuevo_estudiante)
        print("Estudiante registrado correctamente")

    elif opcion == "2":
        mostrar_estudiantes(estudiantes)

    elif opcion == "3":
        nombre = input("Ingrese el nombre a buscar: ")
        estudiante_encontrado = buscar_estudiante(estudiantes, nombre)

        if estudiante_encontrado is not None:
            print("Estudiante encontrado:")
            print("Nombre:", estudiante_encontrado["nombre"])
            print("Edad:", estudiante_encontrado["edad"])
            print("Nota:", estudiante_encontrado["nota"])
        else:
            print("Estudiante no encontrado")

    elif opcion == "4":
        promedio = calcular_promedio_curso(estudiantes)
        print("Promedio del curso:", promedio)

    elif opcion == "5":
        print("Programa finalizado")
        break

    else:
        print("Opción no válida")
```

---

22. Análisis del programa completo

Lista principal

estudiantes = []

Esta lista guarda todos los estudiantes registrados.

Diccionario por estudiante

estudiante = {

```
    "nombre": nombre,
    "edad": edad,
    "nota": nota
}
```

Cada estudiante se guarda como un diccionario.

Funciones

El programa usa funciones para separar tareas:

Función	Tarea
`mostrar_menu()`	Muestra las opciones del sistema
`crear_estudiante()`	Pide datos y crea un estudiante
`mostrar_estudiantes()`	Muestra todos los estudiantes
`buscar_estudiante()`	Busca un estudiante por nombre
`calcular_promedio_curso()`	Calcula el promedio del curso

Ciclo principal

while True:

Este ciclo mantiene el programa funcionando hasta que el usuario elija salir.

Condicional principal

if opcion == "1":

El condicional permite ejecutar una acción diferente según la opción elegida por el usuario.

---

23. Ejemplo con productos

También podemos aplicar la misma lógica a productos.

```
productos = [
    {"nombre": "Mouse", "precio": 12000, "stock": 10},
    {"nombre": "Teclado", "precio": 25000, "stock": 5},
    {"nombre": "Monitor", "precio": 120000, "stock": 3}
]

for producto in productos:
    total = producto["precio"] * producto["stock"]

    print("Producto:", producto["nombre"])
    print("Precio:", producto["precio"])
    print("Stock:", producto["stock"])
    print("Valor total:", total)
    print("-------------")
```

Salida:

```
Producto: Mouse
Precio: 12000
Stock: 10
Valor total: 120000
-------------
Producto: Teclado
Precio: 25000
Stock: 5
Valor total: 125000
-------------
Producto: Monitor
Precio: 120000
Stock: 3
Valor total: 360000
-------------
```

---

24. Ejemplo con productos y condicionales

Mostrar solo productos con bajo stock.

```
productos = [
    {"nombre": "Mouse", "precio": 12000, "stock": 10},
    {"nombre": "Teclado", "precio": 25000, "stock": 2},
    {"nombre": "Monitor", "precio": 120000, "stock": 1}
]

for producto in productos:
    if producto["stock"] <= 2:
        print("Producto con bajo stock:", producto["nombre"])
```

Salida:

```
Producto con bajo stock: Teclado
Producto con bajo stock: Monitor
```

---

25. Ejemplo con función para filtrar productos

```
def mostrar_productos_bajo_stock(productos):
    for producto in productos:
        if producto["stock"] <= 2:
            print("Producto:", producto["nombre"])
            print("Stock:", producto["stock"])
            print("-------------")


productos = [
    {"nombre": "Mouse", "precio": 12000, "stock": 10},
    {"nombre": "Teclado", "precio": 25000, "stock": 2},
    {"nombre": "Monitor", "precio": 120000, "stock": 1}
]

mostrar_productos_bajo_stock(productos)
```

Aquí se combinan:

- Función
- Lista
- Diccionario
- Ciclo
- Condicional

---

26. ¿Cómo pensar un problema antes de programar?

Antes de escribir código, es recomendable responder estas preguntas:

Pregunta 1: ¿Qué datos necesito guardar?

Ejemplo:

Necesito guardar estudiantes.

Entonces puedo usar una lista.

Pregunta 2: ¿Cada elemento tiene varios datos?

Ejemplo:

Cada estudiante tiene nombre, edad y nota.

Entonces cada estudiante puede ser un diccionario.

Pregunta 3: ¿Necesito repetir una acción?

Ejemplo:

Necesito ingresar muchos estudiantes.

Entonces puedo usar un ciclo.

Pregunta 4: ¿Necesito tomar decisiones?

Ejemplo:

Si la nota es mayor o igual a 4.0, está aprobado.

Entonces necesito un condicional.

Pregunta 5: ¿Hay tareas que puedo separar?

Ejemplo:

Registrar, mostrar, buscar y calcular promedio.

Entonces puedo crear funciones.

---

27. Patrón común para resolver problemas

Un patrón muy utilizado en Python es este:

```
def funcion_1():
    pass

def funcion_2():
    pass

datos = []

while True:
    opcion = input("Ingrese una opción: ")

    if opcion == "1":
        pass

    elif opcion == "2":
        pass

    elif opcion == "3":
        break

    else:
        print("Opción no válida")
```

Este patrón sirve para crear programas con menú.

Después reemplazamos los `pass` por instrucciones reales.

---

28. Versión base de menú para practicar

```
def mostrar_menu():
    print("1. Agregar")
    print("2. Mostrar")
    print("3. Buscar")
    print("4. Salir")


datos = []

while True:
    mostrar_menu()

    opcion = input("Seleccione una opción: ")

    if opcion == "1":
        print("Aquí se agregaría un dato")

    elif opcion == "2":
        print("Aquí se mostrarían los datos")

    elif opcion == "3":
        print("Aquí se buscaría un dato")

    elif opcion == "4":
        print("Programa finalizado")
        break

    else:
        print("Opción no válida")
```

Este programa todavía no hace mucho, pero entrega una estructura clara para comenzar.

---

29. Errores comunes al combinar herramientas

Error 1: Confundir lista con diccionario

Incorrecto:

```
estudiante = {
    "nombre": "Ana",
    "nota": 6.5
}

print(estudiante[0])
```

Esto genera error porque `estudiante` es un diccionario, no una lista.

Correcto:

```
print(estudiante["nombre"])
```

Error 2: Intentar acceder a una clave que no existe

```
estudiante = {
    "nombre": "Ana",
    "nota": 6.5
}

print(estudiante["edad"])
```

Esto genera error porque la clave `"edad"` no existe.

Forma segura:

```
print(estudiante.get("edad", "Edad no registrada"))
```

Error 3: Calcular promedio con lista vacía

Incorrecto:

```
notas = []

promedio = sum(notas) / len(notas)
```

Esto genera error porque `len(notas)` es `0`.

Correcto:

```
notas = []

if len(notas) > 0:
    promedio = sum(notas) / len(notas)
    print(promedio)
else:
    print("No hay notas registradas")
```

Error 4: Olvidar agregar datos a la lista

Incorrecto:

```
estudiantes = []

estudiante = {
    "nombre": "Ana",
    "nota": 6.5
}

print(estudiantes)
```

La lista sigue vacía porque no agregamos el diccionario.

Correcto:

```
estudiantes.append(estudiante)
```

Error 5: No usar `return` en una función

Incorrecto:

```
def sumar(a, b):
    resultado = a + b
```

```
total = sumar(5, 3)

print(total)
```

Salida:

```
None
```

Correcto:

```
def sumar(a, b):
    resultado = a + b
    return resultado
```

---

30. Ejercicios propuestos

Ejercicio 1: Lista de números

Crea una lista de números.

Recorre la lista y muestra solo los números pares.

Ejercicio 2: Promedio de notas

Crea una función que reciba una lista de notas.

La función debe retornar el promedio.

Antes de calcular, verifica que la lista no esté vacía.

Ejercicio 3: Registro de estudiante

Crea una función llamada `crear_estudiante`.

Debe pedir:

- Nombre

- Edad
- Nota

Luego debe retornar un diccionario con esos datos.

Ejercicio 4: Lista de estudiantes

Crea una lista vacía llamada `estudiantes`.

Usa un ciclo para ingresar 3 estudiantes.

Cada estudiante debe ser un diccionario.

Agrega cada diccionario a la lista.

Ejercicio 5: Mostrar estudiantes aprobados

Usando una lista de estudiantes, muestra solo aquellos cuya nota sea mayor o igual a 4.0.

Ejercicio 6: Buscar estudiante por nombre

Crea una función que reciba:

- Una lista de estudiantes
- Un nombre a buscar

La función debe retornar el estudiante si existe o `None` si no existe.

Ejercicio 7: Sistema de productos

Crea un programa con menú que permita:

1. Agregar producto.
2. Mostrar productos.
3. Buscar producto.
4. Mostrar productos con bajo stock.
5. Salir.

Cada producto debe tener:

- Nombre
- Precio
- Stock.

Ejercicio 8: Calcular valor total del inventario

Usando una lista de productos, calcula el valor total del inventario.

Fórmula:

valor total producto = precio * stock

El programa debe sumar el valor total de todos los productos.

Ejercicio 9: Menú con funciones

Crea un programa que tenga estas funciones:

- `mostrar_menu()`
- `agregar_estudiante()`
- `mostrar_estudiantes()`
- `buscar_estudiante()`
- `calcular_promedio()`

Usa un ciclo `while` para mantener el menú activo.

Ejercicio 10: Sistema completo de notas

Crea un programa que permita:

1. Registrar estudiantes.
2. Mostrar todos los estudiantes.
3. Buscar estudiante por nombre.
4. Mostrar estudiantes aprobados.
5. Mostrar estudiantes reprobados.
6. Calcular promedio general.
7. Salir.

Cada estudiante debe guardarse como diccionario dentro de una lista.

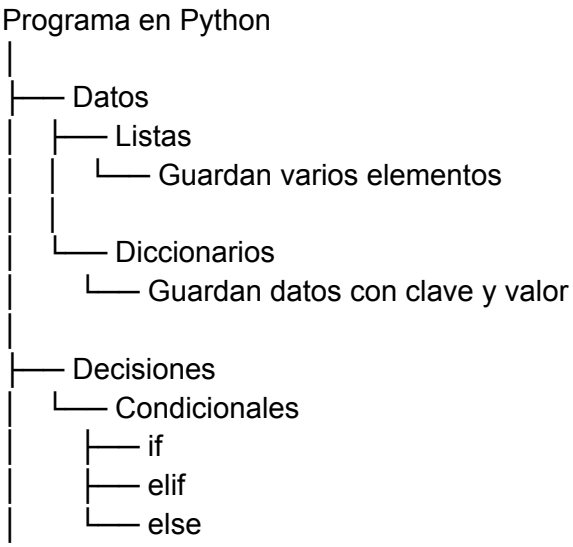
---

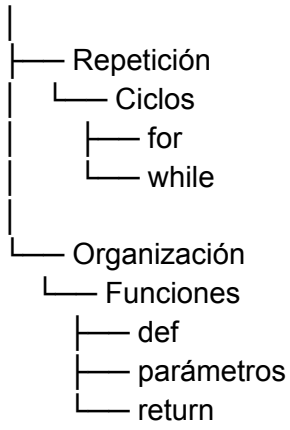
31. Resumen general

Herramienta	Uso principal	Ejemplo
Lista	Guardar varios elementos	`estudiantes = []`
Diccionario	Representar un objeto o entidad	`{"nombre": "Ana", "nota": 6.5}`
Condicional	Tomar decisiones	`if nota >= 4.0:`
Ciclo `for`	Recorrer datos	`for estudiante in estudiantes:`
Ciclo `while`	Repetir mientras se cumpla una condición	`while True:`
Función	Separar y reutilizar código	`def crear_estudiante():`
`append()`	Agregar elementos a una lista	`estudiantes.append(estudiante)`
`return`	Devolver un resultado desde una función	`return promedio`
`break`	Detener un ciclo	`break`

---

32. Mapa mental simple





---

33. Recomendación para estudiantes que están comenzando

Cuando un estudiante se enfrente a un problema, puede pensar de esta forma:

¿Qué datos tengo?
¿Qué datos necesito guardar?
¿Necesito guardar muchos elementos?
¿Cada elemento tiene varios datos?
¿Necesito repetir acciones?
¿Necesito tomar decisiones?
¿Puedo separar el problema en funciones?

Ejemplo:

Necesito registrar productos.

Entonces:

Muchos productos → lista
Cada producto tiene nombre, precio y stock → diccionario
Debo ingresar varios productos → ciclo
Debo saber si hay bajo stock → condicional
Debo organizar el programa → funciones

Cierre

Combinar listas, diccionarios, funciones, condicionales y ciclos es uno de los pasos más importantes para comenzar a construir programas reales en Python.

La clave está en entender qué responsabilidad tiene cada herramienta:

- Las **listas** guardan muchos elementos.
- Los **diccionarios** describen un elemento con varios datos.
- Los **ciclos** permiten repetir tareas.
- Los **condicionales** permiten tomar decisiones.
- Las **funciones** permiten ordenar y reutilizar el código.

Cuando estas herramientas se combinan correctamente, es posible crear programas más completos, organizados y fáciles de mantener.